SEMAINE 6

Autoencodeurs profonds et limites génératives

Encodeur · Latent · Reconstruction · Manifold

S6 — OBJECTIFS (CE QUE VOUS SAUREZ FAIRE À LA FIN)

1. Comprendre l'architecture **encodeur / décodeur** d'un autoencodeur.
2. Expliquer le rôle de la **compression** et du **goulot d'étranglement**.
3. Comprendre intuitivement la notion de **variété** (*manifold*) des données.
4. Expliquer la différence entre **reconstruction** et **génération**.
5. Identifier les **limites génératives** d'un autoencodeur classique.
6. Comprendre pourquoi ces limites motivent les **VAE** étudiés en S7.

1 Fil conducteur : de S5 à S6

MISE EN CONTEXTE — CHANGER DE STRATÉGIE

En S5, nous avons appris à modéliser directement une distribution $p_\theta(x)$ en la factorisant en probabilités conditionnelles. Cette stratégie fournit une vraisemblance exacte, mais la génération reste séquentielle.

*Cette semaine, nous changeons de point de vue : au lieu de générer les composantes de x une par une, nous allons apprendre une **représentation compacte** z de l'observation, puis essayer de reconstruire x à partir de cette représentation.*

QUESTION CENTRALE DE LA SEMAINE

Comment apprendre une représentation compacte z qui conserve l'information essentielle de x ?

2 Introduction : apprentissage non supervisé

Contrairement à l'apprentissage supervisé, où l'on dispose de couples (x, y), un autoencodeur apprend à partir des seules observations x. Il n'a donc pas besoin d'étiquettes y pour définir sa tâche principale.

IDÉE FONDAMENTALE

Un autoencodeur apprend à **reconstruire sa propre entrée** :

$$x \longrightarrow \text{encodeur} \longrightarrow z \longrightarrow \text{décodeur} \longrightarrow \tilde{x} \approx x.$$

Le code latent z doit donc conserver suffisamment d'information pour permettre au décodeur de reconstruire l'entrée.

RECONSTRUCTION $\neq$ GÉNÉRATION

À ce stade, l'objectif du modèle est de reconstruire un x déjà observé. Cela ne signifie pas encore qu'il sait produire correctement un nouvel échantillon à partir d'un latent choisi au hasard.

3 Architecture générale d'un autoencodeur

Un autoencodeur contient deux fonctions apprises :

— **Encodeur** f_ϕ : transforme l'entrée x en une représentation latente z ;

— **Décodeur** g_θ : transforme z en reconstruction $\tilde{x}$.

Les deux étapes de l'autoencodeur peuvent être écrites séparément :

$$z = f_\phi(x), \qquad \tilde{x} = g_\theta(z).$$

Ici, ϕ désigne les paramètres de l'encodeur et θ ceux du décodeur. Cette convention préparera directement la S7 :

$$f_\phi(x) \quad \longrightarrow \quad q_\phi(z \mid x), \qquad g_\theta(z) \quad \longrightarrow \quad p_\theta(x \mid z).$$

L'apprentissage cherche à rendre $\tilde{x}$ proche de x. Il faut donc une quantité qui mesure l'écart entre l'entrée et sa reconstruction. Pour des données continues, une perte simple est l'erreur quadratique :

$$\mathcal{L}_{\text{rec}}(x, \tilde{x}) = \|x - \tilde{x}\|^2.$$

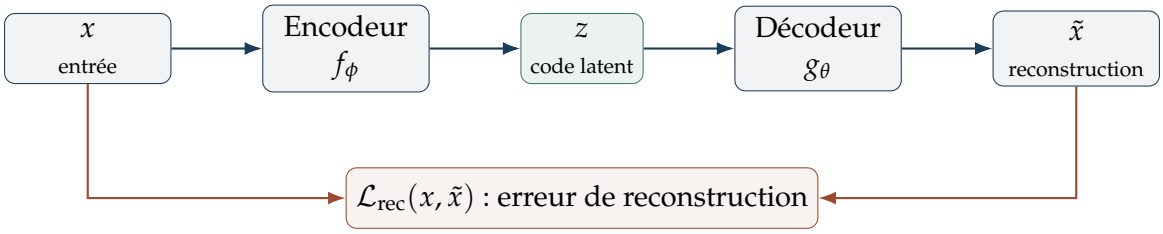
Lecture : plus la reconstruction est proche de l'entrée, plus cette quantité est petite.

CHOISIR UNE PERTE DE RECONSTRUCTION

L'erreur quadratique est fréquente pour des données continues, ou lorsque l'on interprète la sortie du décodeur comme la moyenne d'une reconstruction gaussienne.

Pour des données binaires, ou des pixels normalisés dans $[0, 1]$ interprétés comme probabilités, on peut plutôt utiliser une entropie croisée binaire.

Dans cette séance, la MSE sert de perte simple pour comprendre le mécanisme de reconstruction.



L'encodeur produit une représentation compacte ; le décodeur essaie de reconstruire l'entrée à partir de cette représentation.

DU CONCEPT AU CODE — PROPAGATION AVANT

Le pseudo-code suivant traduit directement les deux équations précédentes : on encode x pour obtenir z, puis on décode z pour produire $\tilde{x}$. La dernière ligne calcule l'erreur de reconstruction utilisée pour l'apprentissage.

```
z = encoder(x)
x_tilde = decoder(z)
loss = mse(x_tilde, x)
```

À observer : le code z est une représentation intermédiaire ; l'objectif d'apprentissage reste la reconstruction de x.

4 Compression et goulot d'étranglement

4.1 Principe

Pour forcer le modèle à résumer l'information, on choisit souvent un espace latent de dimension plus petite que celle de l'entrée. Si D désigne la dimension de x et d celle du latent z, on écrit :

$$x \in \mathbb{R}^D \quad \longrightarrow \quad z \in \mathbb{R}^d, \qquad d < D.$$

Cette zone compacte est appelée **goulot d'étranglement** (*bottleneck*). Lorsqu'il est suffisamment contraint, il limite la capacité du modèle à mémoriser ou recopier l'entrée composante par composante et l'incite à représenter des régularités utiles des données.

NUANCE IMPORTANTE

La seule réduction de dimension ne garantit pas que les coordonnées latentes soient sémantiques, régulières ou directement utilisables pour générer. L'architecture, la perte et les régularisations éventuelles restent importantes.

EXEMPLE CONCRET — MNIST

— une image MNIST contient $28 \times 28 = 784$ valeurs ;
— on choisit par exemple un latent de dimension $d = 20$;
— l'autoencodeur doit reconstruire 784 valeurs à partir de seulement 20 nombres.

4.2 Version linéaire simple

Avant d'introduire des couches profondes, considérons le cas le plus simple : l'encodeur et le décodeur sont deux transformations linéaires avec biais. On obtient alors :

$$z = W_1 x + b_1, \qquad \tilde{x} = W_2 z + b_2.$$

Pour apprendre les paramètres à partir de plusieurs observations, on ne regarde pas l'erreur d'un seul exemple : on calcule l'erreur de reconstruction sur tout le dataset $\mathcal{D} = \{x^{(1)}, \ldots, x^{(m)}\}$, puis on en prend la moyenne :

$$J(\phi, \theta) = \frac{1}{m} \sum_{i=1}^{m} \left\| x^{(i)} - \tilde{x}^{(i)} \right\|^2 .$$

Lecture : on calcule l'erreur de reconstruction pour chaque observation, puis on en prend la moyenne sur le dataset.

LIEN AVEC LA PCA

Dans le cas particulier d'un autoencodeur **sous-complet**, avec :
— des données centrées ;
— un encodeur et un décodeur linéaires ;
— une erreur quadratique ;
le sous-espace appris par l'autoencodeur correspond au sous-espace engendré par les principales composantes de la PCA, à un changement de coordonnées près.

Les autoencodeurs profonds deviennent intéressants lorsque l'on introduit des transformations **non linéaires**, capables de représenter certaines structures que la PCA linéaire ne peut pas capturer.

DU CONCEPT AU CODE — ERREUR DE RECONSTRUCTION

La formule précédente devient, dans un code numérique, une moyenne des différences au carré entre chaque valeur reconstruite et la valeur originale :

```
loss = ((x_tilde - x) ** 2).mean()
```

À observer : ici, le modèle n'optimise pas directement $\log p_\theta(x)$; il minimise une erreur entre l'entrée et sa reconstruction.

5 Manifold learning : intuition géométrique

5.1 Le concept de variété

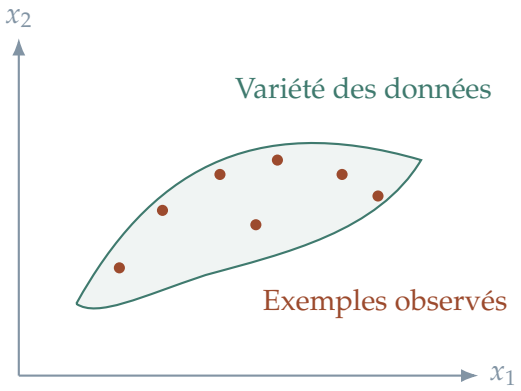
Dans un espace de grande dimension, les données réelles ne remplissent généralement pas toutes les configurations possibles. Elles se concentrent souvent près d'une structure de plus faible dimension.

DÉFINITION INTUITIVE

Une **variété** (*manifold*) est une structure de faible dimension plongée dans un espace de dimension plus grande. Elle sert ici d'intuition pour représenter la région où se concentrent les données plausibles.

ANALOGIE DU BALLON

Une image peut avoir des centaines ou des milliers de pixels. Pourtant, presque toutes les combinaisons possibles de pixels ne ressemblent à aucun objet réel. Les images plausibles occupent seulement une petite région structurée de cet immense espace.



Les points observés occupent une région structurée plutôt que tout l'espace possible.

5.2 Rôle de l'autoencodeur

L'encodeur apprend une carte $x \mapsto z$ et le décodeur une carte $z \mapsto \tilde{x}$. Le latent peut donc être vu comme un système de coordonnées compact décrivant certaines variations présentes dans les données.

PRÉCISION IMPORTANTE

Dire qu'un autoencodeur « apprend le manifold » est une **intuition géométrique utile**, pas une garantie générale. La géométrie effectivement obtenue dépend de l'architecture, de la perte, du bottleneck, des données et des régularisations éventuelles.

6 Autoencodeurs profonds

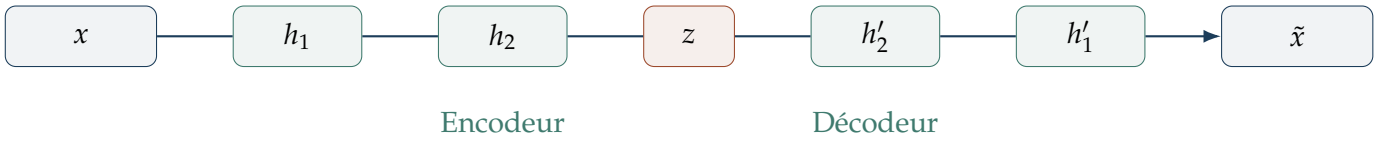
Lorsque les données présentent une structure complexe, une seule transformation linéaire est souvent insuffisante. On empile alors plusieurs couches : les premières construisent des représentations intermédiaires h_1, h_2, puis la représentation est comprimée dans z avant d'être reconstruite dans le sens inverse :

$$x \to h_1 \to h_2 \to z \to h_2' \to h_1' \to \tilde{x}.$$

HIÉRARCHIE DES CARACTÉRISTIQUES

Avec plusieurs couches, le modèle peut construire des représentations de plus en plus abstraites : motifs simples dans les premières couches, combinaisons plus complexes dans les couches profondes, puis représentation compacte dans z.

Cette idée rejoint directement la S1 : les réseaux profonds construisent des représentations intermédiaires hiérarchiques.



Encodeur Décodeur

7 Limites génératives des autoencodeurs classiques

7.1 Ce qu'ils apprennent bien

Un autoencodeur classique est entraîné à reproduire son entrée. Son comportement fondamental peut donc être résumé par la chaîne suivante :

$$x \to z \to \tilde{x}.$$

Il peut donc être très utile pour la compression, la réduction de dimension, le débruitage ou l'apprentissage de représentations.

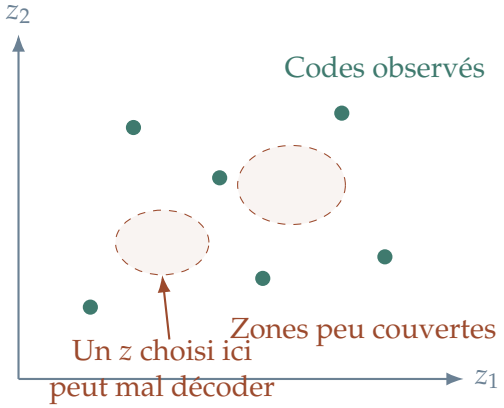
7.2 Ce qui manque pour générer

TROIS LIMITES FONDAMENTALES

1. le modèle n'apprend pas explicitement une densité $p_\theta(x)$;
2. aucun prior $p(z)$ n'est imposé dans l'autoencodeur classique ;
3. le décodeur n'est entraîné que sur les codes produits par l'encodeur, donc des zones arbitraires du latent peuvent être mal apprises.

7.3 Le problème des zones peu couvertes

Pour chaque observation du dataset, l'encodeur produit un code $z = f_\phi(x)$. Le décodeur voit donc principalement ces régions du latent pendant l'entraînement. Rien ne garantit qu'un point choisi arbitrairement entre elles corresponde à une donnée plausible.



INTERPOLATION ENTRE DEUX CODES

Pour tester si l'espace latent est régulier, on peut choisir deux codes z_A et z_B, puis créer un point intermédiaire. Le paramètre α règle la position entre ces deux codes :

$$z_{\text{inter}} = (1-\alpha)z_A + \alpha z_B, \qquad 0 \leq \alpha \leq 1.$$

Lorsque $\alpha = 0$, on retrouve z_A ; lorsque $\alpha = 1$, on retrouve z_B.

Mais, dans un autoencodeur classique, rien ne garantit que tous les points intermédiaires décodent en observations plausibles.

IDÉE À RETENIR

Bonne reconstruction $\not\Rightarrow$ bon modèle génératif.

8 Vers les autoencodeurs variationnels (VAE)

DU MODÈLE DÉTERMINISTE AU MODÈLE PROBABILISTE

	Autoencodeur classique	**VAE (S7)**
Encodeur	$z = f_\phi(x)$	$q_\phi(z \mid x)$
Décodeur	$\tilde{x} = g_\theta(z)$	$p_\theta(x \mid z)$
Latent	Point déterministe	Distribution conditionnelle
Objectif principal	Reconstruction	Reconstruction + structure probabiliste

Les limites précédentes motivent les VAE, étudiés en détail en S7.

APERÇU DE L'IDÉE VAE

Au lieu d'associer chaque observation à un seul point latent, le VAE décrit les latents possibles par une distribution conditionnelle. Cette distribution, notée $q_\phi(z \mid x)$, représente les valeurs de z jugées plausibles pour une observation x :

$$q_\phi(z \mid x).$$

Pour pouvoir échantillonner facilement dans l'espace latent, on cherche aussi à organiser ces distributions autour d'un prior simple, souvent une gaussienne standard :

$$p(z) = \mathcal{N}(0, I).$$

Ici, I désigne la matrice identité.

L'objectif est d'obtenir un espace latent plus structuré et exploitable pour l'échantillonnage.

NE PAS ALLER TROP LOIN EN S6

La dérivation de l'ELBO, la reparamétrisation et l'entraînement complet du VAE appartiennent à la **S7**. Ici, il suffit de comprendre **pourquoi** une structure probabiliste du latent devient nécessaire.

9 Synthèse de la semaine 6 et transition vers S7

SYNTHÈSE — CE QU'IL FAUT RETENIR

— un autoencodeur apprend $x \to z \to \tilde{x}$;

— le bottleneck force une représentation compacte, sans garantir à lui seul un espace latent génératif ;

— les autoencodeurs profonds apprennent des représentations non linéaires ;

— la qualité de reconstruction ne garantit pas un latent bien structuré pour générer ;

— l'absence de prior explicite sur z rend l'échantillonnage arbitraire difficile.

AUTO-VÉRIFICATION

À la fin de cette semaine, vous devriez pouvoir expliquer :

1. le rôle respectif de l'encodeur, du code latent et du décodeur ;
2. pourquoi une faible erreur de reconstruction ne suffit pas pour générer ;
3. ce qu'est un goulot d'étranglement ;
4. pourquoi des zones peu couvertes de l'espace latent posent problème ;
5. ce que le VAE ajoutera à l'autoencodeur classique.

TRANSITION VERS LA S7

"Nous savons maintenant construire un code latent z capable de reconstruire x. Mais une question reste ouverte : comment rendre l'espace latent suffisamment régulier pour que l'on puisse y échantillonner de nouveaux points ?

La S7 introduira les ***Variational Autoencoders (VAE)*** *et ajoutera une structure probabiliste au latent."*

APERÇU DE LA S7

Les deux objets probabilistes qui deviendront centraux en S7 sont donc l'approximation du posterior $q_\phi(z \mid x)$ et le prior $p(z)$:

$$\boxed{q_\phi(z \mid x) \qquad \text{et} \qquad p(z) = \mathcal{N}(0, I)}$$

Objectif : passer d'un espace latent appris pour reconstruire à un espace latent structuré pour générer.